
RL - Reinforcement Learning

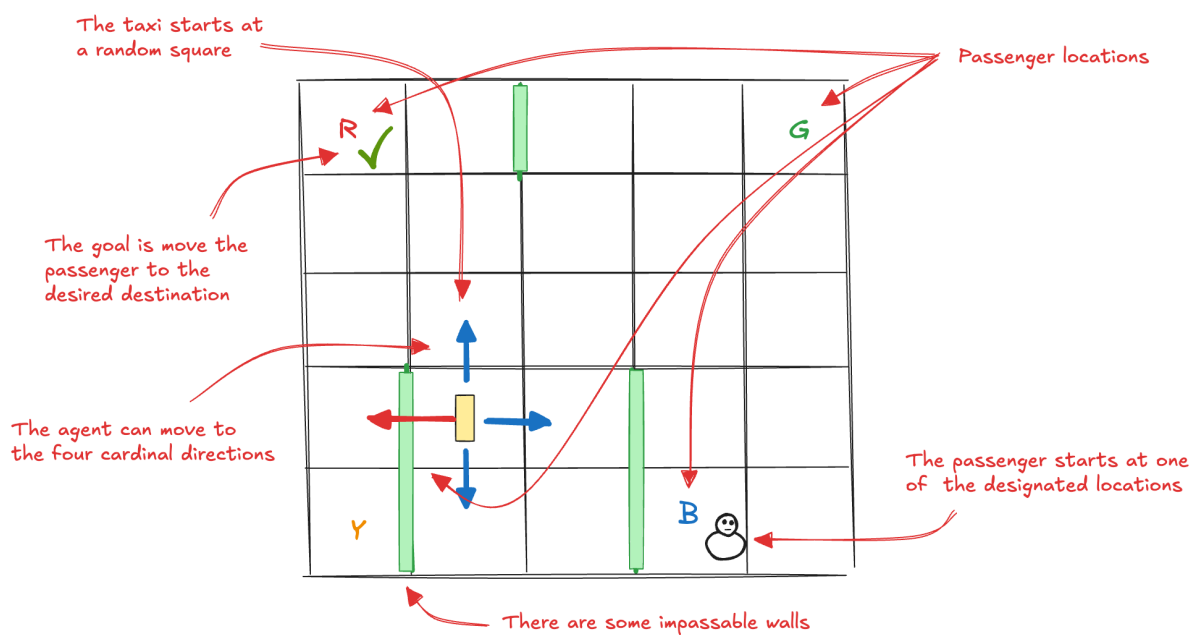
Exercises

Riccardo Berta

2026.05.20

1 Taxi

The Taxi problem involves navigating passengers in a grid world, picking them up and dropping them off at one of four locations (Red, Green, Yellow and Blue) in a 5x5 grid world. The taxi starts off at a random square and the passenger at one of the designated locations. The goal is move the taxi to the passenger's location, pick up the passenger, move to the passenger's desired destination, and drop off the passenger. Once the passenger is dropped off, the episode ends.



There is six possible actions. The agent can move to the four cardinal directions (south, north, east or west) and two other possible actions which are the pick-up or the drop-off:

- 0: Move south (down)
- 1: Move north (up)
- 2: Move east (right)
- 3: Move west (left)
- 4: Pickup passenger
- 5: Drop off passenger

The state is described by the taxi location on the grid (a row and a column number between 0 and 4), a location to drop-off the passenger from four choices, and the passenger which can be in one of the four locations or inside the taxi, in total 500 discrete states. An observation is returned as an int that encodes the corresponding state, calculated by: $((\text{taxi_row} * 5 + \text{taxi_col}) * 5 + \text{passenger_location}) * 4 + \text{destination}$

The player receives positive rewards for successfully dropping-off the passenger at the correct

location. Negative rewards for incorrect attempts to pick-up/drop-off passenger and for each step where another reward is not received.

1 - Import the 'Taxi-v3' grid world from Gymnasium:

```
# You can get the environment from Gymnasium in the same way we got 'Frozen Lake';  
# in order to visually plot the environment you can import it  
# using render_mode="rgb_array"
```

2- Create a random policy as a baseline:

```
# You have to create a function that get in input a state and provide a random action  
# (in the range [0;5])
```

3 - Show the policy in action by rendering the environment several times after different decisions from the random policy:

```
# You have to use a while loop in order to provide the current state to  
# the policy and then make a step in the environment using the action  
# provided by the policy.  
# Try to create a function (to be called "show_policy")  
# to be reused with other policies later.  
# Hints: env.render() provides you an array representing  
# an image of the environment; plt.imshow() can be used to visualize  
# the image on the screen.
```

4 - Write a brute-force function in order to evaluate the probability of success and the average return obtained by a policy, then evaluate the random policy:

```
# You can reuse the "evaluate" function that we already apply to  
# the Frozen Lake environment, however pay attention on how to determine  
# the end of an episode (it is no more state 15).  
# Hints: you have the "done" information from step function.
```

5 - Use the **value-iteration algorithm** to calculate the optimal policy and also the optimal state-value function:

```
# You can reuse the "value_iteration" function that we already  
# apply to the Frozen Lake environment.
```

6 - Calculate the performance of the obtained optimal policy using the brute force approach:

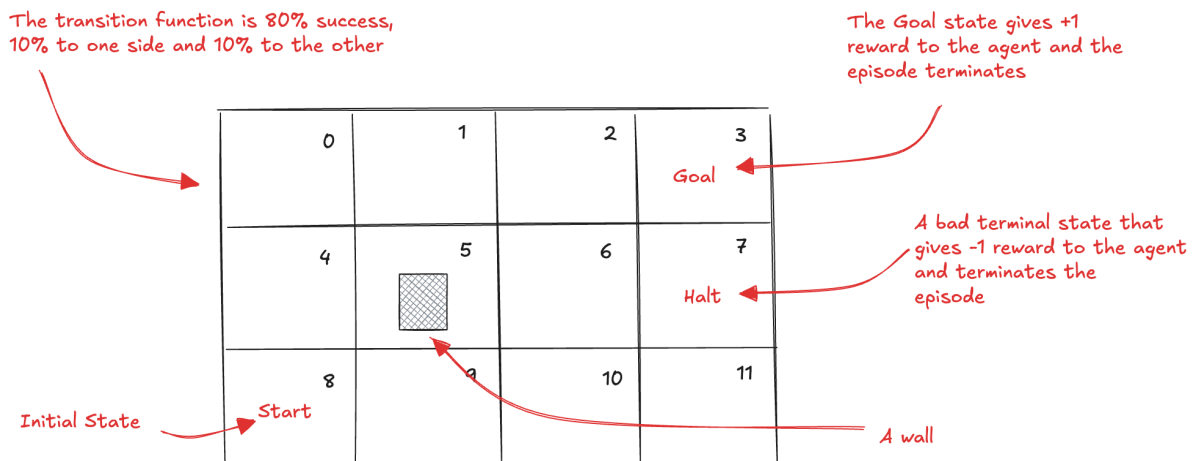
```
# You can reuse the "evaluation" function written before,
# in order to evaluate the optimal policy
```

7 - Show the optimal policy:

```
# You can reuse the "show_policy" function written before,
# in order to show the optimal policy
```

2 Gridworld

Create the environment from the Russell and Norvig's book on AI: **the Gridworld**. This environment is a 3 x 4 grid world in which the agent starts at the bottom-left corner, and it has to reach the top-right corner. There is a hole south of the goal and a wall near the start. The transition function has a 20% noise; that is, 80% the action succeeds, and 20% it fails uniformly at random in orthogonal directions. The reward function is a -0.04 living penalty, a $+1$ for landing on the goal, and a -1 for landing on the hole.



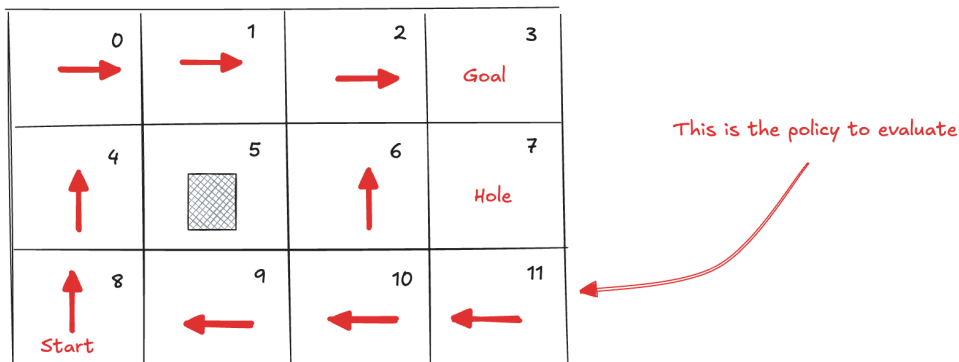
1 - Create the environment Gridworld as described above.

```
# YOUR CODE HERE

# The environment should be a class with the following methods:
# `__init__(self)`: initialize the environment
# `reset(self)`: reset the environment to the initial state
```

```
# `step(self, action)`: take an action in the environment.
# The action should be an integer between 0 and 3, where 0 means left,
# 1 means down, 2 means right, and 3 means up.
# The method should return a tuple (state, reward, done),
# where state is the new state, reward is the reward obtained, and done is
# a boolean indicating whether the episode is finished.
```

2 - Write the following policy for this environment



```
# YOUR CODE HERE

# The policy should be a function that takes an integer between 0 and 11
# (the state) and returns an integer between 0 and 3 (the action).
```

3 - Show the policy by printing the selected action for each state:

```
# YOUR CODE HERE

# The function should print the policy in a human-readable format.
# For example, the output could be:
# state: 0 → action: RIGHT
# state: 1 → action: RIGHT
# etc ...
```

4 - Now, evaluate this policy using $TD(\lambda)$

```
# YOUR CODE HERE

# you need to decay alpha
```

```
# YOUR CODE HERE
```

```
# you need to decay the discount
```

```
# YOUR CODE HERE
```

```
# you need the td_lambda implementation
```

```
# YOUR CODE HERE
```

```
# run the algorithm and print the value function
```

5 - Plot the estimated value for state 8, 6 and 2 for all episodes

```
# YOUR CODE HERE
```

```
# plot the value function over episodes for state 8, 6 and 2
```

3 Blackjack

The game starts with the dealer having one face up and one face down card, while the player has two face up cards. All cards are drawn from an infinite deck (i.e. with replacement). The card values are:

- Face cards (Jack, Queen, King) have a point value of 10
- Aces can either count as 11 (called a "usable ace") or 1
- Numerical cards (2-9) have a value equal to their number

The player has the sum of cards held. The player can request additional cards (hit) until they decide to stop (stick) or exceed 21 (bust, immediate loss). After the player sticks, the dealer reveals their facedown card, and draws cards until their sum is 17 or greater. If the dealer goes bust, the player wins. If neither the player nor the dealer busts, the outcome (win, lose, draw) is decided by whose sum is closer to 21.

There is two possible actions:

- 0: Stick
- 1: Hit

The observation consists of a 3-tuple containing: the player's current sum, the value of the dealer's one showing card (1-10 where 1 is ace), and whether the player holds a usable ace (0 or 1).

The player receives positive +1 rewards for winning the game, negative -1 for loosing the game and 0 reward in case of drawn.

1 - Import the 'Blackjack-v1' environment from Gymnasium:

```
# You can get the environment from Gymnasium in the same way we got 'Frozen Lake'  
# in order to visually plot the environment you can import it  
# using render_mode="rgb_array"
```

2 - Create a random policy as a baseline:

```
# You have to create a function that get in input a state and provide a random  
# action (in the range [0;1])
```

3 - Show the policy in action by rendering the environment several times after different decisions from the random policy:

```
# Reuse the show_policy function we implment for the lab on DP:
```

4 - Write a brute-force function in order to evaluate the probability of success and the average return obtained by a policy, the evaluate the random policy:

```
# You can reuse the "evaluate" function we implment for the lab on DP,  
# however pay attention on how to calculte the success rate, you need to  
# consider that the agent reaches the goal if it obtains a reward of 1 at  
# the end of an episode.
```

5 - Notice that the state space is not a single integer, but a tuple of discrete states. In order to use the state as an index in the value function, you need to convert the tuple of states into a single value.

```
# To convert the tuple of discrete states  
# (Discrete(32), Discrete(11), Discrete(2)) into a single index,  
# you can treat the tuple as a coordinate in a 3D grid and  
# compute a unique index for each combination of values.
```

```
# This is essentially flattening the multi-dimensional space
# into a one-dimensional array.
# Given a tuple of states (s1, s2, s3) where:
# - s1 is in the range 0 to 31 (32 possible values),
# - s2 is in the range 0 to 10 (11 possible values),
# - s3 is in the range 0 to 1 (2 possible values),
# You can calculate the index by assuming that s3 is the least
# significant and s1 is the most significant:

# index = s1 * (11 * 2) + s2 * 2 + s3
```

6 - Use Double Q-learning to calculate the optimal policy and the optimal state-value function:

```
# You can reuse the function we apply to the Slippery Walk environment, in particular
# select_action() to use an epsilon greedy exploration strategy, decay_schedule() to
# calculate learning rate and epsilon values and double_q_learning() to implement the
# algorithm. However, you need to use the state_to_index() function to convert the
# state into an index.
```

7 - Calculate the performance of the obtained optimal policy using the brute force approach:

```
# You can reuse the "evaluation" function written before,
# in order to evaluate the optimal policy
```

8 - Show the optimal policy:

```
# You can reuse the "show_policy" function written before,
# in order to show the optimal policy
```

4 Frozen Lake

Apply and compare SARSA(λ), Q(λ), DynaQ and Trajectory sampling agents on the Frozen Lake (FL) environment using a discount factor $\gamma=0.99$. For the comparison, you can plot only states (0, 6, 10). Get also the optimal state-value functions using Dynamic Programming.

Remember: FL is a simple grid-world environment. It has 16 states and 4 actions. The goal of the agent is to go from a start location to a goal location while avoiding falling into holes. All

transitions landing on the goal state provide a +1 reward, while every other transition in the entire grid world provides no reward. The challenge is that the surface of the lake is frozen, and therefore slippery. So actions have stochastic effects, and the agent moves only a third of the time as intended. The other two-thirds is split evenly in orthogonal directions.

FL is a more challenging environment than Slippery Walk environment. Therefore, one of the most important changes you need to make is to increase the number of episodes the agent interacts with the environment. While in the Slippery Walk environment we allow the agent to interact for only 3.000 episodes, in the FL environment, let your agent gather experience for 10.000 episodes.

1 - Import the FrozenLake environment and extract its MPD dynamics.

```
# YOUR CODE HERE

# You can get the environment from Gymnasium 'FrozenLake-v1';
# In order to visually plot the environment you can import it
# using render_mode="rgb_array"
# You can also import the actions LEFT, DOWN, RIGHT, UP from gym
# Finally, you can import the transition probabilities P from gym
```

2- Create a random policy as a baseline

```
# YOUR CODE HERE

# You have to create a function that gets in input a state and provides a random action
```

3 - Show the policy in action by rendering the environment several times after different decisions from the random policy:

```
# YOUR CODE HERE

# You have to use a while loop in order to provide the current state to
# the policy and then make a step in the environment using the action
# provided by the policy.
# Try to create a function (to be called "show_policy")
# to be reused with other policies later.
# Hints: env.render() provides you an array representing
# an image of the environment; plt.imshow() can be used to visualize
# the image on the screen.
```

4 - Solve the environment using Dynamic Programming:

5 - Print the values of the optimal state-value function

```
# YOUR CODE HERE
```

```
# Copy the print_state_value_function function from the lecture notes and use it to  
# print the state-value function for the best policy
```

6 - Implement the SARSA(λ) algorithm and use it against the FL environment

```
# YOUR CODE HERE
```

```
# Copy the decay_schedule function from the lecture notes
```

```
# YOUR CODE HERE
```

```
# Copy the decay_discounts function from the lecture notes
```

```
# YOUR CODE HERE
```

```
# Copy the select_action function from the lecture notes
```

```
# YOUR CODE HERE
```

```
# Copy the sarsa_lambda function from the lecture notes and modify it in order to ask  
↪ the environment the number  
# of states and actions following the Gymnasium interface (.n), and extract the id of  
↪ the starting state from the first  
# output of the reset() function.
```

```
# YOUR CODE HERE
```

```
# Apply sarsa(lambda) for 10.000 episodes and gamma = 0.99
```

7 - Plot the state-value functions for states (0, 6, 10) for SARSA(λ) and compare with the optimal values. Then observe how the agent struggle to estimate the optimal state-value function.

```
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_sarsa_lambda for each episode
↳ for states 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10
```

8 - Implement the Q(λ) algorithm and use it against the FL environment

```
# YOUR CODE HERE

# Copy the q_lambda function from the lecture notes and modify it in order to ask the
↳ environment the number
# of states and actions following the Gymnasium interface (.n) and extract the id of
↳ the starting state from the first
# output of the reset() function.
```

```
# YOUR CODE HERE

# Apply q(lambda) for 10.000 episodes ans gamma = 0.99
```

9 - Plot the state-value functions for states (0, 6, 10) for Q(λ) and compare with the optimal values. Then observe as Q(λ) agent is moving the estimates of the optimal state-value function toward the true values, unlike SARSA(λ). Now, to be clear, this is a matter of the number of steps, also SARSA(λ) would converge to the true values if given more episodes.

```
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_q_lambda for each episode for
↳ states 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10
```

10 - Implement the DynaQ algorithm and use it against the FL environment.

```
# YOUR CODE HERE

# Copy the dyna_q function from the lecture notes and modify it in order to ask the
↳ environment the number
# of states and actions following the Gymnasium interface (.n) and extract the id of
↳ the starting state from the first
# output of the reset() function.
```

```
# YOUR CODE HERE

# Apply DynaQ for 10.000 episodes and gamma = 0.99
```

11 - Plot the state-value functions for states (0, 6, 10) for DynaQ and compare with the optimal values.

```
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_dq for each episode for states
↳ 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10
```

12 - Implement the Trajectory Sampling algorithm and use it against the FL environment.

```
# YOUR CODE HERE

# Copy the trajectory_sampling function from the lecture notes and modify it in order
↳ to ask the environment the number
# of states and actions following the Gymnasium interface (.n) and extract the id of
↳ the starting state from the first
# output of the reset() function.
```

```
# YOUR CODE HERE

# Apply trajectory_sampling for 10.000 episodes and gamma = 0.99
```

13 - Plot the state-value functions for states (0, 6, 10) for Trajectory Sampling and compare with the optimal values.

```
# YOUR CODE HERE

# import matplotlib.pyplot as plt and plot the V_track_ts for each episode for states
↪ 0, 6 and 10
# draw a horizontal line for the optimal_V for states 0, 6 and 10
```

14 - Plot the state-value function mean absolute error for all the methods.

```
# YOUR CODE HERE

# Copy the moving average function from the lecture notes
```

```
# YOUR CODE HERE

# plot the moving average of the absolute error between the optimal_v
# and the v for each algorithm
```

15 - Show the optimal policy:

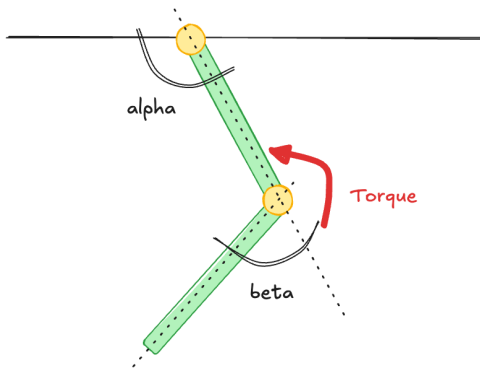
```
# YOUR CODE HERE

# You can reuse the "show_policy" function written before,
# in order to show the optimal policy
```

4.1 Acrobot

The Acrobot is a classic problem in control theory. The environment consists of two links connected linearly to form a chain, with one end of the chain fixed. The joint between the two links is actuated. The goal is to apply torques on the actuated joint to swing the free end of the linear chain above a given height while starting from the initial state of hanging downwards.

- The action space is discrete, deterministic, and represents the torque applied on the actuated joint between the two links: (0) apply -1 torque, (1) apply 0 torque to the actuated joint, (2) apply 1 torque to the actuated joint
- The observation space is a 6-dimensional space with the sine and cosine of the two joint angles and their angular velocities.



The goal is to have the free-end reach a designated target height in as few steps as possible, and as such all steps that do not reach the goal incur a reward of -1. Achieving the target height results in termination with a reward of 0. The reward threshold is -100.

1 - import the environment from Gymnasium and show it with a random policy

```
# YOUR CODE HERE

# You can get the environment from Gymnasium 'Acrobot-v1';
# In order to visually plot the environment you can import it
# using render_mode="rgb_array". Get also the state size
# and action_size.
```

2 - Show the policy in action by rendering the environment several times after different decisions from a random policy:

```
# YOUR CODE HERE

# You can use the same function already used for the cartpole environment to
# implement a random agent and to show the policy.
```

3 - Create the evaluation function for a policy

```
# YOUR CODE HERE

# It is the code to run the policy several times and show the
# average reward.
```

4 - Create a neural network as a function approximator for the Q-function

```
# YOUR CODE HERE
```

```
# Add the code for a neural network using the PyTorch library and  
# the code of an optimizer as in the cartpole example
```

5 - Create a policy that is greedy with respect to the Q-function

```
# YOUR CODE HERE
```

```
# Write a function that, given a state, returns the action  
# that maximizes the Q-function.
```

6 - Add the code for the experience buffer

```
# YOUR CODE HERE
```

```
# Add the code for the experience buffer as in the cartpole example
```

7 - Add the code of the exploration strategy based on epsilon-greedy method:

```
# YOUR CODE HERE
```

```
# Add the code for an exploration strategy based on the epsilon-greedy algorithm
```

8 - Add the code to train the neural network using the collected experiences:

```
# YOUR CODE HERE
```

```
# Re-use the optimize method as in the cartpole example
```

9 - Write the code of the NFQ algorithm

```
# YOUR CODE HERE
```

```
# Re-use the optimize method as in the cartpole example
```

10 - Apply NFQ to the environment

```
# YOUR CODE HERE

# Apply NFQ to the acrobot environment
```

11 - Evaluate the learned policy

```
# YOUR CODE HERE
```

12 - Compare performance of random policy vs the one learned by the neural network

```
# YOUR CODE HERE

# Run the evaluation on the two policies and compare
```

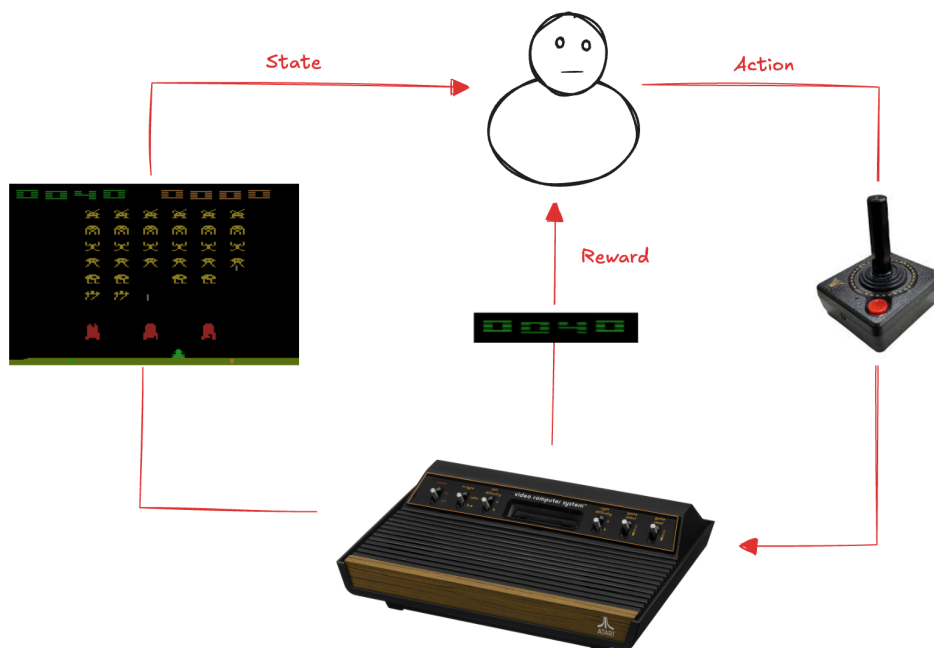
13 - Show the policy behavior

```
# YOUR CODE HERE

# You can show the policy learned by the agent using the show_policy function
```

4.2 Atari Games

Atari 2600 is a video game console which provides several popular games such as pong, space invaders, pacman, break out, centipede and many more. Gymnasium includes ALE-py, the **Arcade Learning Environment (ALE)** which is a simple framework that allows researchers to develop AI agents for Atari 2600 games. It is built on top of the **Stella emulator** and separates the details of emulation from agent design. The main Gymnasium page provides a list of all the Atari environments. The objective of this notebook is to **train an agent to play a game of the Atari console** using the DQN algorithm **learning from raw pixel data**, like in the famous paper V. Mnih, et al. **"Playing Atari with Deep Reinforcement Learning"**, NIPS 2013:



In particular, we can consider **the classic Pong game**. The game is played like tennis. Using a controller, each player hits the ball by moving vertically the paddles on the game field. A player scores one point when the opponent misses a hit. The first player who scores 21 points wins the game. The action space is discrete and deterministic with 6 actions (not all meaningful for Pong): 0 (noop), 1 (fire), 2 (right) and 3 (left), 4 (right fire), 5 (left fire). The observation space is the screen image (210x160x3). The state space is really high-dimensional and to use it you need to modify the neural network architecture adding some convolutional layer in order to extract features from the image.

1 - Define a variable with the name of the Atari game you want to solve (e.g. Pong) and create an instance of the environment.

```
# YOUR CODE HERE

# Import Gymnasium and create an instance with "render_mode" set to "rgb_array". You
# need to import also "ale_py" plugin and to register it with Gymnasium
↳ (gym.register_envs(ale_py)).
# Set also the parameter "frameskip" to 1. Later, we will see that we want to control
# the number of frames to skip as a parameter of our code.
```

2 - Create a function to show the game in action by rendering the environment and selecting actions at random or using a policy.

```
# YOUR CODE HERE
```

```
# Define a function "random_pi" that takes as input a state and a Q-function and  
# returns a random action.
```

```
# YOUR CODE HERE
```

```
# Define a function "show_policy" that takes as input an environment,  
# a policy and a Q-function and shows the policy in action for a number of episodes
```

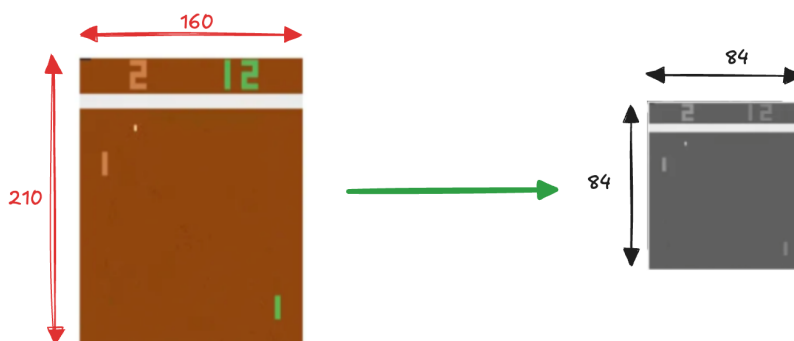
3 - Use the previous function in order to see one episode of the game with a random policy.

```
# YOUR CODE HERE
```

```
# Just call the "show_policy" function defined above passing the environment, None as  
↪ agent and  
# the number of episodes you want to visualize
```

4 - We would like to use as state the raw pixels data. However, **directly feeding the raw game screen image is not efficient**, since it will be **computationally expensive**. So we need to pre-process the environment output to reduce the dimensionality of the problem. Moreover the Atari environments have some game specific issues, which are not relevant for the learning process, that we need to manage. In order to do that, you can use the **wrapper function**, which is a convenient way to modify an existing environment without having to alter the underlying code directly. Using wrappers will allow you to avoid a lot of boilerplate code and make our environment more modular. Here a list of the useful modifications that we can apply to the Atari environment:

1) The raw game screen size is 210x160 pixels with three color channels (RGB), however we don't need the color information and we can resize the image in order to reduce the dimensionality of the problem.



2) Atari games typically run at 60 frames per second, however not every frame contains crucial information for decision-making. To reduce computation and memory requirements, and to speed up training, we can skip a certain number of frames. Moreover, some games have a flickering effect (when the game draws different portions of the screen on even and odd frames) due to the old Atari platform's limitation. For the human eye, such quick changes are not visible, but they can confuse a neural network. So, we can take the maximum of every pixel in the last two frames and use it as observation.

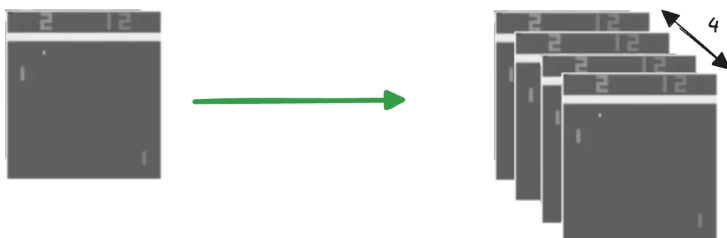
3) In order to inject variability into the initial state of each episode, promoting exploration and helping the agent discover a more robust policy across different starting conditions, we can introduce a certain number of "no-operation" actions at the beginning of each episode in order to add stochasticity to the initial conditions of the environment.

Add a wrapper in order to apply all those modifications to the environment.

```
# YOUR CODE HERE

# Gymnasium provides a wrapper to apply those pre-processing to the
# observations for the Atari games, you can use the "AtariPreprocessing" wrapper
# from "gymnasium.wrappers" module.
```

5 - A single raw screenshot of the game used as state is not enough to extract the information about the game state. How can the agent learn to play just seeing a single image at a time? It is **not possible to build an idea about how the game objects are moving**. In Pong, just looking at a single frame don't provide enough information to understand if the ball is moving left to right or right to left. For example, in the Cart-pole example, the state is composed by the position of the cart, the angle of the pole, but it includes also the velocity of both. To deal with this, we need to consider as state a series of frame and not just one. In this way, the agent can learn how the objects are moving. So, we need to apply a wrapper that will stack the last N (e.g. 4) frames together and use them as state.



```
# YOUR CODE HERE

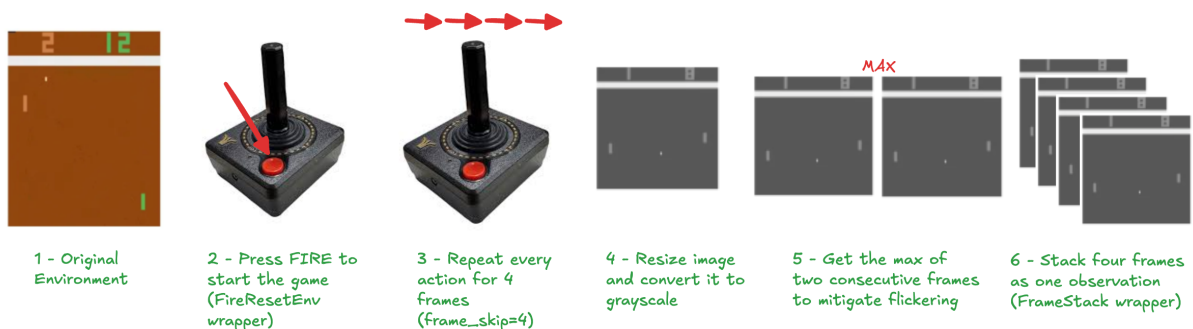
# Gymnasium provides the "FrameStackObservation" wrapper to stack the last n frames
↳ of the game. Apply it to the
# environment.
```

6 - Some games, as Pong, require a user to press the FIRE button to start the game, we need to take this action on reset. Add another wrapper to take this action on reset.

```
# YOUR CODE HERE

# This wrapper can be found in the stable_baselines3 library. It is called
↳ "FireResetEnv"
# Import it and apply it to the environment.
```

7 - Finally, we write a "make_atari_env()" function that takes as input the name of the environment and returns the wrapped environment



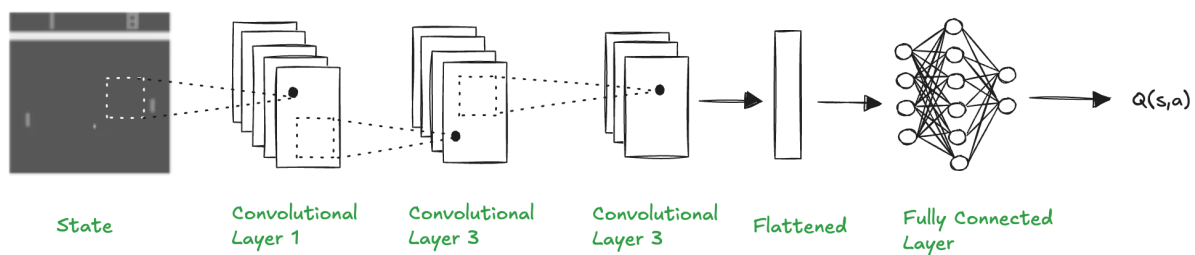
```
# YOUR CODE HERE

# You create an environment and then apply in order all the wrappers needed
```

8 - Since we are dealing with images, **instead of using a fully connected neural network, this time we need to use a convolutional neural network.** The convolutional layers extract the features from the image and output the feature maps, then the flattened feature map can be fed to the fully connected layer which returns the Q value. You can implement the following architecture:

- Convolutional layer with 32 filters, kernel size 8, stride 4, padding 0, followed by a ReLU activation function

- Convolutional layer with 64 filters, kernel size 4, stride 2, padding 0, followed by a ReLU activation function
- Convolutional layer with 64 filters, kernel size 3, stride 1, padding 0, followed by a ReLU activation function
- Flatten layer
- Fully connected layer with 512 units, followed by a ReLU activation function
- Fully connected layer with a number of units equal to the number of actions



```
# YOUR CODE HERE
```

```
# Add the code for the neural network using the PyTorch library. And
# create two networks, one for the online network and one for the target network.
```

9 - Add the Replay Memory to store the transitions (state, action, reward, next_state, done).

```
# YOUR CODE HERE
```

```
# Add the code for the replay buffer already used presented in the DQN implementation.
# We need a larger replay buffer because the problem is more complex.
```

10 - Create a the code for exploration (decaying epsilon-greedy policy) and exploitation (greedy policy) and the code for doing the evaluation of the agent.

```
# YOUR CODE HERE
```

```
# Add the code for decaying epsilon and the epsilon-greedy policy as already
# done for the DQN implementation
```

```
# YOUR CODE HERE
```

```
# Add the code for an evaluation of the agent as already
# done for the DQN implementation.
```

11 - Add the code for the optimization of the Q-function based on the experience sampled from the replay buffer. Remember to use the target network to compute the target Q-values.

```
# YOUR CODE HERE

# Add the code of the optimization algorithm as already
# done for the DQN implementation. This time you need to pay attention
# to states which are a stack of four images
```

12 - Write the code of the DQN algorithm (exploiting the new convolutional network architecture). You should modify the code in order to save the network weights after some episodes.

```
# YOUR CODE HERE

# Add the code for the DQN algorithm, exploiting the convolutional neural architecture
# already defined, the replay buffer as already implemented in DQN algorithm.
# At the beginning, if a model file is already present on the disk, you should load the
# weights and
# sometimes, you should save the model on the disk.
```

14 - Apply the algorithm to the ATARI environment. Pay attention, the following code takes a lot of time to run (days on a powerful machine).

```
# YOUR CODE HERE

# Apply the previous written algorithm to the ATARI environment, please pay attention
# to the parameters.
```

15 - Smooth the result using a sliding window

```
# YOUR CODE HERE

# smooth the result using a sliding window
```

16 - Show the score curve as a function of the number of episodes

```
# YOUR CODE HERE
```

```
# Show the moving average reward during the training process
```

18 - Show the agent playing the game using the greedy policy

```
# YOUR CODE HERE
```

```
# You can show the policy learned by the agent using the "show_policy" function
```